

WarehouseFlow Project Management Summary

Project Overview

WarehouseFlow is a solo distributed systems project that simulates a warehouse management system routing pipeline:

- **ingestion-service** receives HTTP orders and publishes them to Kafka
- **routing-service** consumes orders, routes them to warehouses, updates inventory, and persists audit decisions
- Redis models per-warehouse state
- Postgres stores routing decisions for correctness checks and experiment analysis
- Terraform provisions the AWS deployment stack
- Locust, Prometheus, Grafana, and the dashboard support experimentation and presentation

This document extends the original project plan through the final week and shows how the project moved from initial design to final state.

Ownership

This was an individual project, so all workstreams were owned and implemented by Nithish Kadam Ganesh.

Workstream	Owner	Initial Goal	Final State
Core backend services	Nithish	End-to-end order flow	Completed
AWS / Terraform infrastructure	Nithish	Deployable cloud stack	Completed to validated plan
Load testing and experiments	Nithish	3 experiments	Expanded to 7 experiments
Observability and dashboard	Nithish	Basic metrics	Prometheus, Grafana assets, and standalone interactive dashboard
Final report and demo assets	Nithish	Submission-ready	Completed

Initial Design To Final State

Original Design

The project began with four main goals:

1. Build a horizontally scalable order-ingestion and routing system.
2. Model warehouse state independently so failures could be injected and observed.
3. Measure real distributed systems tradeoffs through experiments rather than only describing them.
4. Produce a final artifact set that showed both engineering depth and project process.

Final Implemented System

The final system includes:

- Go ingestion service with request validation, UUID assignment, Kafka publish, and Prometheus metrics
- Go routing service with region-aware warehouse selection, atomic inventory decrement, picker assignment, audit logging, and multiple concurrency strategies
- Resilience toolkit with circuit breaker, bulkhead, retry with jitter, fail-fast timeouts, and graceful degradation buffer
- Local development stack using Docker Compose with Redpanda, Redis, Postgres, Prometheus, and Grafana
- AWS Terraform stack for VPC, ALB, ECS, ElastiCache, RDS, SQS, ECR, and attempted MSK deployment
- Seven experiments with saved CSV outputs and generated chart images
- Standalone interactive dashboard for demo/video use

Week-By-Week Execution

Week 1: Baseline System Online

Planned:

- Create Docker Compose environment
- Implement ingestion and routing services
- Stand up Redis/Postgres/Kafka-based flow
- Add a smoke-test path

Completed:

- `docker-compose.yml` created for Redpanda, 3 Redis nodes, Postgres, Prometheus, and Grafana
- `ingestion-service` implemented with `POST /api/v1/orders` and metrics
- `routing-service` implemented with Redis-backed inventory and Postgres audit logging
- Terraform scaffold created for AWS deployment
- Unit tests added for handlers and core routing logic

Exit criterion:

- One order could flow from HTTP -> Kafka -> routing -> Postgres

Week 2: Scaling and Observability

Planned:

- Build Locust-driven workload
- Create dashboards and chart pipeline
- Validate horizontal scaling behavior

Completed:

- Experiment 1 load profile created
- Prometheus and Grafana assets added
- Horizontal scaling results recorded in `results/experiment1`

- Week 2 analysis documented

Exit criterion:

- Throughput and latency trends measurable as worker/task count changed

Week 3: Failure and Contention

Planned:

- Inject warehouse failures
- Add DLQ-focused resilience checks
- Compare contention control strategies

Completed:

- Failure injection and recovery scripts added
- Optimistic and pessimistic routing engines added
- Experiment 2 and 3 runners and datasets completed
- DLQ and zero-oversell reasoning documented
- Week 3 analysis completed

Exit criterion:

- Failure response and contention tradeoffs measurable with supporting evidence

Week 4: Final Analysis and Polish

Planned:

- Expand experiment coverage
- Finish final writeup and demo assets
- Improve presentation quality

Completed:

- Added four more experiments: network partition, Kafka partitioning, noisy neighbor, and cold start
- Added resilience package and unit tests
- Added interactive dashboard in [dashboard/](#)
- Generated chart images for all seven experiments
- Final report, integration guide, improvement roadmap, and demo script completed

Exit criterion:

- Full submission packet ready with working local demo path, reproducible experiments, and final analysis

Task Breakdown By Area

Core Backend

- Stabilized ingestion API contract
- Kept Kafka payload schema consistent across services

- Preserved routing correctness with atomic Redis operations
- Added unit tests around validation, routing preference, rejection paths, and resilience behavior

Infrastructure

- Created full AWS stack in Terraform
- Validated Terraform syntax and plan in AWS
- Adjusted Terraform to reuse `LabRole` for ECS in learner-lab style accounts
- Attempted full apply; creation progressed through VPC, ALB, RDS, ElastiCache, ECR, ECS cluster, and SQS before failing on MSK permission
- Cleaned up all partially created AWS resources after the MSK permission failure

Experiments

- Stored all experiment CSVs under `results/experiment*/`
- Experiment charts under `results/charts/`
- Cross-linked implementation details with experiment results in docs

Problems Encountered And How They Were Handled

Problem	Why It Mattered	Resolution
Running two Go modules separately created coordination overhead	The ingestion and routing services are separate Go modules, so tests and dependency cleanup had to be run in both places	Treated each service as its own deployable unit and verified <code>ingestion-service</code> and <code>routing-service</code> independently
One resilience retry test failed	Test suite was not fully green for submission	Fixed retry default handling so <code>MaxDelay</code> and jitter fall back correctly when omitted
Kafka added local startup timing issues	The ingestion service could start before Redpanda was fully healthy, causing temporary publish/connectivity errors during early smoke tests	Added health checks in Docker Compose and waited for Redpanda readiness before running the services and smoke test
End-to-end debugging crossed several systems	A failed order could be caused by HTTP validation, Kafka publish, consumer lag, Redis inventory, picker availability, or Postgres logging	Added audit rows in Postgres and Prometheus metrics so failures could be narrowed down by checking each pipeline stage
Redis inventory correctness was subtle under concurrency	A normal read-then-write flow could oversell inventory under simultaneous orders	Moved the correctness boundary into Redis with a Lua atomic check-and-decrement script

Problem	Why It Mattered	Resolution
Picker assignment could become a hidden bottleneck	Even if inventory existed, a warehouse should not accept unlimited concurrent work without available pickers	Modeled pickers as a Redis queue using LPOP and RPUSH , which serialized picker assignment naturally
Failure behavior was different for crashed vs slow warehouses	A crashed Redis node fails quickly, but a slow node can create high tail latency and cascading goroutine buildup	Added resilience patterns, especially circuit breaker, bulkhead, fail-fast timeouts, and retry with jitter
AWS deployment failed on kafka:CreateCluster	Full cloud deployment blocked at MSK creation	Documented blocker, confirmed plan validity, and destroyed partial infrastructure to avoid cost drift
Solo-project scope was larger than planned	Backend, infrastructure, load tests, experiments, dashboards, and reports all had to be completed by one person	Prioritized the core flow first, then added experiments and dashboard polish only after the end-to-end pipeline was working
Live dashboard integration was out of scope	Connecting the dashboard directly to Kafka, Redis, Postgres, and live experiments would add significant complexity and could make the demo fragile	Built a standalone simulator that visually explains the backend behavior and experiment findings, while the actual backend and experiment data remain in the Go services, load tests, CSVs, and charts